

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Design, brute-force validation, and a controlled comparison with Q3C and pgSphere

J. Salgado¹  and M. Delli Veneri¹ 

SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield SK11 9FT, United Kingdom
jesus.salgado@skao.int, michele.delliveneri@skao.int

Received ...; accepted ...

ABSTRACT

Context. Astronomical archives answer positional queries from relational databases, and in the PostgreSQL world two extensions dominate: Q3C, which keys each source by an integer drawn from a quad-tree on a cube and stores it in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree. The first is compact and fast for points; the second is general and indexes regions as well.

Aims. We ask whether the two can be combined: a single index that keeps the integer key and the B-tree, yet answers cones, polygons, cross-matches and stored footprints in archives’ own geometry language, and whether it can beat each established extension on its own ground.

Methods. We present **skycell**, a PostgreSQL extension that keys positions by their order-29 HEALPix cell and turns a query region into index ranges. How finely to cut is decided by a cost model – the price of an extra index range against the rows it removes – with local source density read from the histogram **ANALYZE** already keeps. Cells are classified conservatively against the region, and a planner support function rewrites the predicate into B-tree range conditions plus an exact test, or, for a constant cone, a custom scan walks the covering’s ranges itself; a companion GiST opclass family serves stored regions directly. Coverings are validated by brute force against millions of catalogue points, including adversarial cones and polygons. Performance is measured against Q3C 2.0.5 and pgSphere 1.5.2 on 10- and 50-million-row corpora – synthetic, resampled from the real Gaia DR3 density field, and real DR3 positions – in trials where all three methods answer the same query back to back in randomized order, analysed paired, warm and cold.

Results. Warm, skycell’s custom scan beats pgSphere at every radius from 1'' to 3° on all three corpora: by 17–33% on the synthetic two and 4–31% on real Gaia DR3 positions, where the range rewrite alone loses below 3°; cold it is faster or level at every radius. Its index is 214 MB against pgSphere’s 685 MB – 38 GiB against 160 GiB at Gaia DR3 scale for the pgSphere-plus-Q3C pair covering the same ground – builds 30 times faster, answers convex polygons 2.7–4.4 times faster, cross-matches 20–33% faster than **q3c_join** (faster in all 24 cells of a sweep on real positions) and 2.9–3.8 times faster than pgSphere, and its row estimates are wrong by a factor 1.2 rather than 2.5–3.0. Its one free cost parameter is derived from the relation’s own statistics, its chosen cell order costs about 3% against the best available one, and its density estimate is good to within a factor 1.6 on ordinary sky (low by 10–100× in a globular cluster core, which costs the covering chosen at most a factor 1.8). At 50 million rows it stays 14–31% faster warm at every radius and its cold advantage widens, to a factor 2.7 at 1°. A **float4** GiST box key matched to pgSphere’s own representation beats every region-opclass alternative tried, including pgSphere’s native GiST, on every corpus measured. An attempt to sharpen the covering’s own search further looked like a clean win in execution time alone but reversed under honest total-time accounting – reported as a methodological caution, not shipped as a default.

Conclusions. It can. A cost-based covering over an equal-area integer key, answered by its own scan node, beats pgSphere at cone and polygon searches and Q3C at cross-matching – each at its own speciality – from one index a third the size, while speaking ADQL’s geometry vocabulary and holding circles and polygons in one column. That makes it a strong choice for an archive’s positional layer; what remains is planning cost on small relations.

Key words. Astronomical databases: miscellaneous – Catalogs – Virtual observatory tools – Methods: data analysis

1 Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory these arrive as ADQL (Ortiz et al. 2008) through the Table Access Protocol (Dowler et al. 2019), and the service must answer them with an index.

Two PostgreSQL extensions carry most of this

traffic. Q3C (Koposov & Bartunov 2006) projects the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve, so each source is one 64-bit integer in a B-tree. pgSphere (Chilingarian et al. 2004) adds spherical types indexed with GiST over bounding volumes, so regions can be stored and queried as data. The scale this has to work at is set by archives already in operation: the ESA Gaia Archive (Salgado et al. 2017, 2020) serves order 10^9 sources and lists cross-match

among its server-side capabilities (Sect. 6.5). Neither idea is new to astronomy – the Hierarchical Triangular Mesh (Kunszt et al. 2001) already keyed positions by a subdivided polyhedron, and the zones algorithm (Gray et al. 2006) pushed a purely relational arrangement for cross-matching – but catalogue size has grown and most queries now arrive through a generated layer (e.g. Demleitner et al. 2014) rather than being hand-written.

The two designs trade off: a linear key in a B-tree is compact and cheap to probe, but a region must become a set of key ranges first, and that approximation’s quality depends on how finely the region is cut; a tree of bounding volumes needs no such step but has larger, overlapping keys and imposes no order on the table. This paper presents `skycell`, which keeps the integer key and asks a cost question at the point where the classical approach fixes a constant: how finely should a region be cut? The answer depends on two quantities the database already knows – local source density and the price of an extra index range – and is computed rather than configured.

The measurements are built to survive scrutiny rather than flatter the method: every query is answered by all three methods back to back in randomized order, analysed paired; everything is measured warm and cold; and one corpus is density-matched to the real Gaia DR3 sky while another is real, unresampled Gaia DR3 positions, not a density field standing in for them. Sect. 2 develops the B-tree index, Sect. 3 the GiST alternative for stored regions, Sect. 4 what is proven versus tested, Sect. 5 the ADQL surface, Sect. 6 the benchmark and results (source-like catalogues, cross-matches, stored regions), and Sect. 7 what this establishes and what remains open.

2 The B-tree index

2.1 An equal-area integer key

A position is keyed by its order-29 HEALPix cell in the nested scheme (Górski et al. 2005) ($12 \cdot 4^{29} \approx 3.5 \times 10^{18}$ cells of about 0.4 mas), computed by the standard `ang2pix_nest` arithmetic and stored as a signed 64-bit integer – a fixed-order index, not a multi-order key, so a B-tree orders keys along the HEALPix curve. Three properties follow, only the first shared with Q3C’s cube quad-tree. Cells are *nested*: the order-29 descendants of a cell p at order k are exactly the contiguous interval

$$[p \cdot 4^{29-k}, (p+1) \cdot 4^{29-k} - 1], \quad (1)$$

so a region covered by cells of any orders becomes a union of integer intervals a B-tree answers with range scans (the MOC standard’s NUNIQ encoding (Fernique et al. 2014), used for stored regions, Sect. 3, packs the order into the key as $4 \cdot 4^k + p$, related by Eq. (1)). Cells are *equal area*, so keys-per-interval is proportional to source density per

solid angle: the `ANALYZE` histogram is therefore a crowding map (Sect. 2.2) with no extra machinery. And cells are *standard* – the same numbering underlies Gaia’s `source_id` (Gaia Collaboration 2016) and IVOA MOC maps (Fernique et al. 2014) – so keys, coverings and footprints are interoperable rather than private to one extension.

2.2 Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer false positives and more ranges. With cells of size s covering a region of radius r and local source density ρ ,

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}), \quad (2)$$

both terms in rows: the second is the expected false positives, and c_{range} converts one additional index range into the false-positive rows it is worth avoiding – a property of the installation (Sect. 6.4 measures it). A covering follows the region’s boundary, so for $s \ll r$ the ranges grow as r/s while the wasted area grows as the perimeter times s ; writing $A_{\text{cover}} \simeq \pi(r + \beta s)^2$ and $n_{\text{ranges}} \simeq \gamma r/s$ and minimising Eq. (2) gives

$$s_{\star} = \sqrt{\frac{\gamma c_{\text{range}}}{2\pi\beta\rho}}, \quad (3)$$

independent of region size: sky density and range price alone decide how finely to cut. For $s \gtrsim r$ the balance differs, so rather than solve the branches the implementation evaluates Eq. (2) at all 30 orders and takes the cheapest, letting a range cap and a cell-size guard simply remove candidates. $\beta = 0.7$ and $\gamma = 2$ are geometric; Sect. 6.4 shows the order choice is flat enough near its optimum not to need them exact.

The same 0.25° cone covered in two places on the same catalogue illustrates both regimes: at high Galactic latitude, 11 sources leave almost nothing to exclude, so the model buys one range of large cells (5.6 times the cone’s area); in the bulge, 1183 sources make the false positives cost more than extra ranges save, so the model cuts two orders deeper (23 cells, 8 ranges, 1.3 times the area).

Two guards keep the model safe against its own inputs: a partly covered cell is never allowed to exceed a multiple (64 by default) of the region’s own area, bounding the damage from a dense cluster the histogram underestimates; and gaps between ranges are merged when the rows they contain cost less than the range they save.

The density estimate. PostgreSQL’s `ANALYZE` stores, for the indexed expression, an equi-depth histogram: a sorted list of $B+1$ bounds $h_0 < h_1 < \dots <$

h_B in key space such that each bucket $[h_i, h_{i+1})$ holds about N/B of the table's N rows. Because the key is an equal-area cell index, the rows in a key interval $[a, b]$ are estimated by linear interpolation within the buckets it spans,

$$\hat{n}(a, b) = \frac{N}{B} \sum_i \frac{|[a, b] \cap [h_i, h_{i+1})|}{h_{i+1} - h_i}, \quad (4)$$

and the corresponding density in rows per steradian is

$$\hat{\rho}(a, b) = \frac{\hat{n}(a, b)}{(b - a + 1) A_{29}}. \quad (5)$$

The estimate used by Eq. (3) is $\hat{\rho}$ evaluated over the order- m cell containing the region's centre, with m chosen so that $s_m \approx 4r$: large enough to contain enough rows to be a density, small enough to be local.

Three caveats: the quantity is a density, not the planner's own selectivity (computed separately from the same histogram); Eq. (4) assumes rows are spread uniformly inside a bucket, which fails for a cluster smaller than one, hence the area guard of Sect. 2.2; and the estimate is only as local as a bucket (with $B = 1000$, $N = 10^7$, a bucket holds 10^4 rows). Section 6.4 measures how close $\hat{\rho}$ lands to the truth.

Classifying cells: what is bounded and what is not. A cell must be classified against the region as inside, outside or straddling its boundary, and the three outcomes are not symmetric: a cell called *outside* is dropped, so if that is ever wrong the query silently loses rows, while *inside* and *straddling* cells both produce a superset the exact predicate (Sect. 2.3) trims back. Only the outside decision therefore needs a bound rather than an estimate.

The bound used is

$$R_{\text{cell}}(k, p) = \min(\text{maxpixrad}(k), \max_i \angle(\mathbf{c}_{k,p}, \mathbf{v}_i)), \quad (6)$$

where $\mathbf{c}_{k,p}$ is the cell's centre, $\mathbf{v}_i$ its corners, and the first term the analytic worst case the reference HEALPix implementation uses over the whole sphere. A cell is outside when $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ for a cone of centre $\mathbf{q}$, radius r (correspondingly per bounding half-space for a polygon); $\epsilon = 2 \times 10^{-13}$ rad covers accumulated floating-point error, five orders of magnitude below the leaf cell's resolution. The inside test is tighter, using distances to the cell's own corners and edge midpoints directly. A tempting outside-test alternative (distance to the chords between corners) is not a bound – it can under-estimate by a factor reaching 159 near order 18, invisible to every test we had until it wasn't; Eq. (6) avoids it, at a cost of 2.5% more ranges and no measurable time.

2.3 Planner integration and ingestion

The extension is written in C against the PostgreSQL 15+ server API: a geometry layer with

no database dependencies and a layer connecting it to the planner via a *planner support function* (`SupportRequestSimplify`, the mechanism PostGIS uses for `STDWithin`). Before path generation, a predicate such as `point <@ circle(...)` – any region, unless it is a constant cone, which has a scan of its own (below) – is replaced by a disjunction of range conditions on the key (`cell BETWEEN lo1 AND hi1 OR ...`) conjoined with an exact test, kept only if the relation carries an index on the position. The index conditions are ordinary range predicates, so the planner estimates their selectivity from the same histogram, and the exact test's own argument carries the covering's occupied fraction, keeping the product of the two estimates close to the truth. Two caches matter for planning cost: the density model and index identity per backend, and coverings memoised by region and statistics source (worth 45–70 and up to 50 μs per plan on a TAP corpus). For joins, where the region is not constant, the rewrite emits a fixed number of range slots per outer row, as `q3c_join` does; the custom scan below covers each outer row's cone instead, and Sect. 6.5 shows the difference matters.

A custom scan for cones. The rewrite leaves the ranges to PostgreSQL, which builds, estimates and costs an index path for every arm of the disjunction: about 10 μs per range here, more than the covering, and enough to decide a small cone warm (Sect. 6.4). A cone is therefore answered by a planner custom scan instead (`skycell.custom_scan`, the default; heap tables). The support function leaves the cone as one clause and supplies its selectivity (the covering's expected rows times the cone's share of its area); a path hook offers one custom path wherever a B-tree on the cell expression can serve it, costed as PostgreSQL costs the same work. The node walks the covering in one of two modes, chosen from the column's correlation as PostgreSQL chooses between an index and a bitmap scan: one index scan per range in cell order when the heap follows the cell order, or every range's TIDs into one bitmap, each heap page read once, when it does not; rows are rechecked with the exact test. Since no range is planned, a range is priced as the scan pays for it (Eq. 8). A cross-match cone, whose centre comes from another relation's row, gets the node parameterized by that row on the inner side of a nested loop, covering each row's cone with the ranges it needs; its row estimate is measured at plan time on 32 sampled outer rows, since a density model cannot know that cross-match targets sit on catalogue sources. Generic plans, correlated sub-selects and the Q3C-shaped spellings (`skycell.join`, `skycell.radial.query`) take the same paths; a cone under an `OR`, and every non-cone region, keep the rewrite. A regression suite checks the same rows with the scan on and off, in both modes, on PostgreSQL 16 and 18.

Ingestion and maintenance. The index is an ordinary B-tree on an expression: one 8-byte key per insert, no method-specific rebalancing. Because the covering depends on the histogram, statistics matter more than for a GiST index – the area guard of Sect. 2.2 bounds the damage from a stale one, and ANALYZE on the 10-million-row corpora here takes 0.5–1.4 s.

3 The region index

Regions as stored objects. A footprint is covered by a small number of cells written in the NUNIQ encoding of the MOC standard and stored one per row in an ordinary B-tree. The footprints containing a given position are those whose cell is one of that position’s ancestors, a short list of equality lookups; refinement stops when cells become negligible next to the region, keeping the ancestor list short.

skyregion is one type internally tagged by kind rather than a family of per-shape types, so a single footprint column holds circles and polygons together – an ObsCore **s_region** column, say, mixes single-pointing circles with non-trivial polygons in the same rows, covered by one function regardless of kind. pgSphere has no equivalent: **scircle** and **spoly** are separate types, so a mixed-shape column needs two columns in its place.

3.1 A GiST opclass family for stored regions

A **skyregion** column can carry its own GiST index instead of the B-tree rewrite above, serving **&&** and both containment operators directly, keyed either by a small number of spherical caps (exact for a circle at any radius, but larger) or a plain axis-aligned 3D box. Shrinking the box key from **float8** (48 bytes) to outward-rounded **float4** (24 bytes, pgSphere’s own GiST representation) is sound because recheck is already **true** regardless of key precision: a looser box costs false positives the exact test removes, never rows. It is now the default region opclass; the multi-cap key remains better for a column mixing very different region scales, where box’s looseness around a large circle costs more than its smaller key saves. Measured against pgSphere’s native GiST on isolated single-scale bands, a mixed circle/polygon corpus, and the real **fpr** catalogue across all four region strategies: Sect. 6.6 (Table 10).

The same shrink does *not* transfer to a GiST key over *points* (**skypos_cap4_gist_ops**, shrinking only the internal-node cap, since a point’s leaf stores its position exactly and leaf exactness is load-bearing there): leaf entries dominate a GiST tree’s page count, and the leaf layer here has nothing to shrink, so no win results – the structural reason the region shrink is safe (recheck already **true**) does not hold

for points. Numbers in Sect. 6.6.

4 What is proven, and what is tested

Proposition. Let R be a cone or convex polygon and let $\mathcal{C}$ be the covering computed for it. If (i) every point of a HEALPix cell at order k lies within $\text{maxpixrad}(k)$ of that cell’s centre, and (ii) every angle computed by the classification carries an absolute error below ϵ , then every order-29 cell that intersects R lies in one of the ranges of $\mathcal{C}$.

Proof sketch. The descent starts from cells whose union is the whole sphere and, at each step, keeps a cell, drops it, or replaces it by its four children (whose union is the parent), so the kept union shrinks only where a cell is dropped, which happens only when classified outside. With $R_{\text{cell}} = \text{maxpixrad}(k)$, (i) bounds every point of cell (k, p) within R_{cell} of its centre, so (ii) and the triangle inequality on the sphere make $\angle(\mathbf{q}, \mathbf{c}_{k,p}) - R_{\text{cell}} > r + \epsilon$ sufficient for the whole cell to miss the cone; the polygon case applies the same argument per bounding half-space. Truncation (range cap, area guard, step budget) only stops refinement, keeping a cell whole, so it cannot remove one either – a cell containing a point of R intersects R and is never dropped. $\square$

The proposition is conditional: (i) is a property of HEALPix, not of this work (the reference maxpixrad is analytic, verified over 1.6×10^6 sampled boundary points, largest observed ratio 0.9999; **skycell.exact_cells = off** rests on (i) alone, at the cost of Sect. 2.2); (ii) is an assumption about floating-point arithmetic, with ϵ chosen five orders of magnitude above the plausible accumulated error and five below the leaf cell.

Because a covering that misses a cell silently loses rows, the implementation is also checked by exhaustion: the HEALPix layer against known centres, round trips at all 30 orders, the nesting property, equal area ($\chi^2/\nu = 1.14$ over 768 cells), and the bounds of Sect. 2.2, over 1.6×10^6 sampled points including poles and zone boundaries; the covering against a 2-million-point clustered catalogue for cones to 120° and convex polygons, 30 000 adversarial cones (poles, zone boundary, face edges, radii $0.02 \text{ mas} - 10^\circ$) with 200 points each, and 7266 adversarial polygons with 866 537 points – all asserting zero false negatives, the test that caught the error of Sect. 2.2. Two SQL-level regression suites require every indexed answer to equal a sequential scan with the exact predicate (240 cones, 60 polygons, both join forms, MOC lookup; and the ADQL surface, index-plan presence, epoch propagation and frame conversions).

Table 1: What a translator emits, for the ADQL constructs where the pgSphere-plus-Q3C pair and `skycell` differ most (the remaining surface – BOX, DISTANCE, AREA/CENTROID, epoch propagation – translates the same way and is omitted for space). `skycell` uses ADQL’s own spellings throughout; pgSphere needs region types and Q3C a separate fast path per shape.

ADQL	pgSphere + Q3C		skycell
POINT/ CIRCLE/ POLYGON	spoint/scircle/spoly		point/circle /polygon
CONTAINS(POINT, CIRCLE)	q3c_radial_query pos <@ scircle	or	point <@ circle
INTERSECTS(r ₁ , r ₂)	r ₁ && r ₂ (no Q3C re- gion type)		r ₁ && r ₂
cross-match	q3c_join; 2.0–3.8× (Sect. 6.5)	pgSphere slower	skycell_cone or skycell_join in the join

5 The ADQL surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so a translator emits the standard’s spelling rather than a dialect, where pgSphere needs `spoint/scircle/spoly` and Q3C a separate fast path per shape with no region type at all; the operator form `pos <@ circle(...)` is the one a planner can answer from an index. Table 1 sets `skycell` against the pgSphere-plus-Q3C pair for the constructs where they differ most.

`skyregion`’s text representation is IVOA STC-S (Rots 2007) (`CIRCLE ICRS  $\alpha$   $\delta$  r / POLYGON ICRS ...`), the same syntax an ObsCore `s_region` column already carries, so such a column casts directly from, and prints back as, the archive’s own text with no translation step. For hand-written SQL migrating from Q3C, `skycell_radial_query` and `skycell_join` take Q3C’s own argument lists as a prefix replacement, both reducing to the canonical `skycell_cone`.

`skycell` also implements the IVOA UDF registry’s epoch-propagation and frame functions (`ivo_epoch_prop`, `ivo_apply_pm`, `ivo_healpix_*`) with the rigorous propagation of the Hipparcos catalogue (ESA 1997), so a proper-motion-aware cone search is an indexable cone widened by the largest motion of interest, intersected with an exact test on the propagated position – exercised by round-trip tests (Barnard’s star to sub-microarcsecond over 16 years) rather than benchmarked.

6 Tests

6.1 Three corpora

Three corpora, each answering an objection the previous one invites. The *designed* corpus is 10 million synthetic sources with a density contrast resembling a real optical survey (35% uniform, 45% Galactic disk, 12% bulge, 8% in 200 clusters) – chosen by the method’s own authors. The *Gaia* corpus answers that without real positions: source counts from the ESA Gaia archive for every order-9 HEALPix cell (3 145 728 cells of 0.11°, summing to the published DR3 total) give 10 million positions placed proportionally to each cell’s count, the real sky’s crowding down to that map scale, Poisson below it by construction. The *real* corpus removes the remaining objection: ten million real Gaia DR3 positions, plus 1.6 million sources forming eight *complete* fields at full density (ω Cen, 47 Tuc, M4, M13, Baade’s Window, the Galactic centre, the LMC, and an off-cluster control), since thinning alone dilutes crowding to 0.55% of its true surface density – used for the density-estimator and ablation measurements (Sect. 6.4), where sub-cell structure is the whole question.

6.2 Protocol

Q3C 2.0.5, pgSphere 1.5.2 and `skycell` each hold the same logical rows in their own natural layout, all three physically ordered along a space-filling curve so heap locality is held equal (Table 2); row counts are compared on every query. The measurement is organised as *trials*: one query centre and radius, all three methods answering it back to back in an order drawn at random (and recorded, so an order effect can be measured rather than assumed away), repeated five times with the query set traversed in random order each time. The analysis is paired – median over repetitions per method, ratio between methods for that query, so its own difficulty cancels – with a percentile bootstrap (4000 resamples) interval; a ratio is called a difference only when its interval excludes 1. Everything is measured *warm* (an untimed pass precedes the measurement) and *cold* (server restarted, caches empty, first repetition only), with a separate randomized EXPLAIN (ANALYZE, BUFFERS) pass recording planning time, execution time, buffer pages and row estimate apart from each other. Cone radii span 1'' to 3° with 16–400 queries each, half from catalogue positions and half uniform on the sky; cross-matches join a 200 000-row probe table to the catalogue, half displaced by $\sim 0.3''$; footprint queries ask which of 20 000 stored circles contain each probe.

6.3 Reproducibility

Table 2 records the two configurations in enough detail to repeat or contest the measurements.

Table 2: The measured configurations. The second host ran every custom-scan measurement, with the range rewrite alongside in the same trials; the first ran the rest, before the custom scan existed. Rows spanning both columns are common to the two.

Item	First host	Second host
Machine	Apple M3 Pro laptop (aarch64), 11 cores; Docker Desktop Linux VM, 8 GB shared with other containers; NVMe SSD	cloud micro-VM (KVM, x86-64), 4 vCPU Intel Xeon 2.8 GHz, 15 GB; one virtual disk, its backing storage not visible
PostgreSQL	18.6, PGDG package	18.6, built from source (gcc 13.3, -O2, no LLVM)
Settings	<code>shared_buffers</code> 2 GB, <code>effective_cache_size</code> 5 GB, <code>random_page_cost</code> 1.1, <code>work_mem</code> 64 MB, JIT off, parallelism off	
Extensions	Q3C 2.0.5, pgSphere 1.5.2, skycell 0.6 (range rewrite)	the same Q3C and pgSphere built from source; skycell 0.26 (custom scan and range rewrite)
Range price	Eq. (7), derived per relation, 116 here	Eq. (8) for the custom scan, 34 on the Gaia catalogue; Eq. (7) for the rewrite, 110
Used for	every table and figure not listed opposite	Tables 3, 4, 6, 7, 8 and 9, Fig. 1a,b
Rows	10^7 per corpus (5×10^7 for the Gaia corpus's scale test, on both)	
Schema	each extension's own B-tree/GiST on its own key (<code>q3c_ang2ipix</code> , <code>spoint</code> , <code>cell_int8</code>), heap ordered by its own key; <code>VACUUM (FREEZE)</code> and <code>ANALYZE</code> (stats target 1000) before measuring	
Repetitions	5 (warm), 1 (cold); 1456 queries per repetition	
Uncertainty	percentile bootstrap, 4000 resamples, over queries	

What this protocol still does not control.

Two machines, each with one storage device, sharing one configuration – absolute times are properties of a machine, only the comparisons travel. The custom scan was measured only on the second, and wherever it is compared with the range rewrite the two ran in the same trials there, so that comparison does not cross hosts; comparing it with results from the first host does. The three extensions are compared as configured here: a different Q3C formulation or pgSphere index configuration could move their numbers (Sect. 6.4 reports one such effect dominating Q3C's small-cone results).

6.4 Source-like catalogues

Cone searches. Table 3 and Fig. 1a give the primary result: paired ratios with intervals, on both corpora, warm and cold.

Every column is *skycell*'s custom scan, its default (Sect. 2.3). Warm, it is faster than pgSphere at every radius on all three corpora, every interval excluding 1: by 17–33% (ratios 0.67–0.83) on the two synthetic ones and by 4–31% on real positions (Sect. 6.4). The range rewrite, run in the same trials on the same host, is 0.88–1.07 of pgSphere up to 30' and 0.69–0.91 at 1° and 3° – where the original measurement on the host of Table 2 put it – so the gain is the custom scan's, not the host's: 0.78–0.84 of the rewrite's time up to 30', 0.87–0.97 above (`bench/results-pg18/rewrite-vs-custom.txt`). At 1'' it plans in 0.036 ms against the rewrite's 0.055, still above pgSphere's 0.029, and wins in execution, 0.022 ms against 0.040, reading 4.1 pages on average to pgSphere's 5.5. This matters more than it sounds: for the range rewrite on the host of Table 2, measuring one method's whole grid then the next instead gives ratios of 0.69–0.87 below 30' where the paired protocol finds 0.89–1.03, since the phase-ordered arrangement flatters whichever method runs later – here, *skycell*. The paired number is the one we stand behind.

Cold, the custom scan is faster at every radius on both synthetic corpora except 1', where the two are level (0.96–0.99): by 12–21% at 1'', 10'' and 6', 25–31% at 30' and a factor 1.6–2 at 1° and 3°, because it touches a third of pgSphere's buffer pages at 1° (on average 59 against 155 on the designed corpus, 85 against 282 on the Gaia one) – a few microseconds of memory traffic when those pages are warm, the whole cost of the query when they are not. Cold ratios depend on the storage under the page cache: a cold read here costs about 0.1 ms, a cache beneath the virtual machine rather than disk latency, so this is cold for PostgreSQL and the virtual machine, and with true disk latency every page saved would count for more.

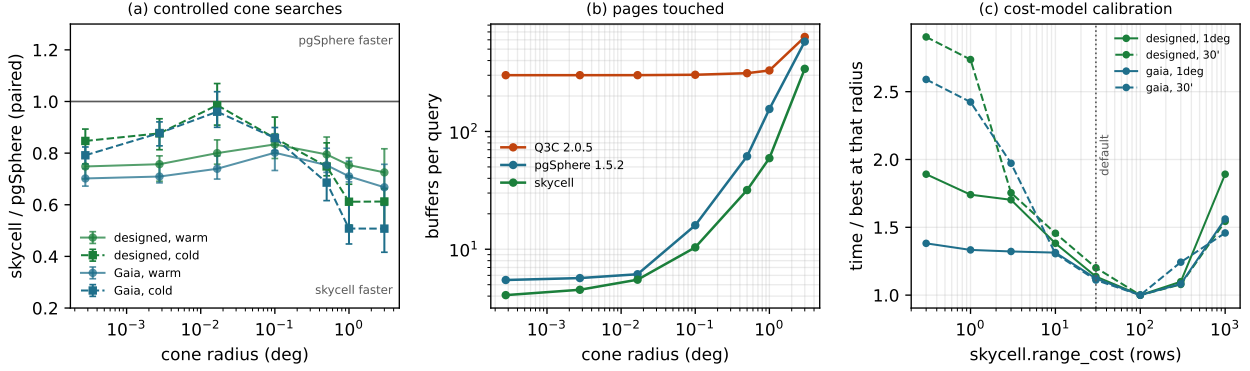


Figure 1: (a) Paired ratio of **skycell** to **pgSphere** against cone radius, 95% bootstrap intervals, both corpora and cache regimes; below the line **skycell** is faster. (b) Buffer pages touched per query (mean, designed corpus): at 1° **skycell** reads a third of **pgSphere**’s pages. (a) and (b): **skycell**’s custom scan, the trials of Table 3’s synthetic columns; (c): the range rewrite on the host of Table 2. (c) The cost model’s one free parameter, swept over four decades and normalised to the best setting per radius; flat-bottomed over a decade either side of its minimum, why deriving it (Sect. 6.4) costs nothing in speed.

On real Gaia DR3 positions the result holds.

The resampled corpus places positions uniformly within each 0.11° map cell; the real one keeps every scale of clustering the map cannot see. On 10 million real DR3 positions (Table 3’s rightmost columns) the custom scan is faster than **pgSphere** warm at every radius, by 4% at 6’ to 31% at 3°. It plans in at most 0.004ms more than **pgSphere** and executes in 0.52–0.82 of its time, touching fewer pages at every radius (4.3 against 5.7 at 1”, 90 against 313 at 1°). The range rewrite, run in the same trials, is 1.27–1.56 of **pgSphere** from 1” to 30’ and 1.25 at 1° – the small-cone deficit an earlier revision of this paper reported from the first host – and the cause is its planning, 0.014–0.17ms over **pgSphere**’s against a small cone that **pgSphere** answers in about 0.05ms in total; the custom scan is 0.57–0.70 of the rewrite’s time there. Real sub-cell clustering does not cost **skycell** extra rows fetched: it cost the rewrite one planned path per range, which the custom scan does not spend. Cold (Table 3, last column), the two methods read separate copies of the same rows, so neither warms the other’s pages; every centre is new and kept apart from every other, and the server is restarted before each method’s pass at each radius. From 6’ up **skycell** is faster by a factor 1.5–2.1 (0.48–0.67), reading 0.54–0.79 of **pgSphere**’s pages; it is 0.87 at 1” and level at 10” and 1’, where a first-touch query reads two or three pages and one fewer page read is worth about as much as the remaining difference.

Against Q3C, **skycell** is faster by a factor 1.7–50, a comparison of *implementations*: **q3c_radial_query** 2.0.5 expands every call into 100 key ranges regardless of radius, about 300 buffer pages before any row is read. Against **q3c_join**, intended for cross-matching and carrying no such fixed

expansion, **skycell**’s join is 20–33% faster on resampled and real positions alike (Sect. 6.5) – informative about the two indexing *approaches*; about the cube quad-tree as a method we make no claim, since a covering-aware radial query in Q3C would plausibly close most of the gap. Neither method was given prepared statements or plan reuse, which would help **skycell** most. The order effect the randomization controls is small but real, inflating variance rather than biasing the ratios.

Does the advantage grow with the catalogue?

Ten million rows is 0.55% of Gaia DR3, so we repeated the protocol on a 50-million-row Gaia-density corpus, where **pgSphere**’s 3439 MB GiST index exceeds the 2GB **shared_buffers** while **skycell**’s 1071 MB B-tree still fits (Table 4). The advantage holds at every radius and grows where pages decide. Warm, the custom scan is 14–31% faster than **pgSphere** at every radius at 5×10^7 rows (0.69–0.86): from 30’ up the margin holds (0.69–0.75) as the buffer gap widens (188 pages against **pgSphere**’s 642 at 1°), while at an arcminute and below it narrows a little (0.70–0.74 to 0.78–0.86), since both methods touch five or six pages at any table size and **skycell**’s planning grows with density (0.036 to 0.042ms at 1”). Cold, the gain grows with size from 30’ up, to a factor 1.9 at 30’, 2.7 at 1° and 2.4 at 3°, and holds below (0.81–0.98). The range rewrite, run in the same trials, reproduces the first host’s 5×10^7 measurement in direction (0.94–1.09 below 30’, 0.68–0.80 above); the custom scan’s gain over it is at small radii, 0.81–0.85 of its time below 30’, and nil from 1° up, where both read the same pages. Size costs **skycell** nothing at any radius and widens its lead for the queries whose pages stop fitting in memory.

Table 3: Cone searches: paired ratio of **skycell** to pgSphere, 95% percentile-bootstrap interval over queries. Below 1 means **skycell** is faster. **Bold/italic**: interval excludes 1, faster/slower; plain: not distinguishable. n : distinct queries at that radius, each answered five times per method (warm) or once (cold), all methods randomized per trial (the skycell/Q3C ratio, 0.02–0.58 depending on radius and cache regime, is given in the text, Sect. 6.4). All columns: **skycell**’s custom scan, its default (Sect. 6.4), PostgreSQL 18.6 in a 4-core cloud container (Table 2, second host). *Designed* and *Gaia*: **bench/results-pg18/**; cold is the server stopped and the container’s page cache dropped before a single pass. *Real*: 10 million real Gaia DR3 positions (**random_index** < 10^7), its own centre draw at the same radii and counts, **bench/results-pg18/realcone/**. Warm: one shared heap, per-method blocks in two orders. Cold: one table per method, fresh centres, server restarted before each method’s pass at each radius, first touch only. Cold everywhere is cold for PostgreSQL and the virtual machine but not for the host’s own cache (Sect. 6.4).

radius	n	designed corpus		Gaia corpus (resampled)		real Gaia DR3 positions	
		warm	cold	warm	cold	warm	cold
1''	400	0.75 [0.70, 0.79]	0.85 [0.80, 0.89]	0.70 [0.67, 0.74]	0.79 [0.76, 0.83]	0.83 [0.81, 0.84]	0.87 [0.82, 0.94]
10''	400	0.76 [0.69, 0.79]	0.88 [0.81, 0.94]	0.71 [0.68, 0.76]	0.88 [0.83, 0.92]	0.85 [0.82, 0.86]	1.01 [0.94, 1.09]
1'	300	0.80 [0.76, 0.85]	0.99 [0.91, 1.07]	0.74 [0.70, 0.79]	0.96 [0.90, 1.04]	0.90 [0.87, 0.92]	0.98 [0.90, 1.04]
6'	200	0.83 [0.78, 0.88]	0.86 [0.80, 0.94]	0.80 [0.74, 0.86]	0.86 [0.81, 0.90]	0.96 [0.93, 0.99]	0.61 [0.55, 0.65]
30'	100	0.79 [0.75, 0.86]	0.75 [0.71, 0.84]	0.75 [0.73, 0.82]	0.69 [0.62, 0.76]	0.81 [0.78, 0.86]	0.67 [0.60, 0.72]
1°	40	0.75 [0.71, 0.78]	0.61 [0.52, 0.68]	0.71 [0.69, 0.77]	0.51 [0.45, 0.72]	0.80 [0.75, 0.88]	0.53 [0.44, 0.66]
3°	16	0.73 [0.66, 0.82]	0.61 [0.50, 0.66]	0.67 [0.63, 0.76]	0.51 [0.41, 0.60]	0.69 [0.64, 0.75]	0.48 [0.43, 0.54]

Table 4: The same protocol at two catalogue sizes on the Gaia density field: paired ratio of **skycell**’s custom scan to pgSphere, PostgreSQL 18.6, second host (Table 2); at 5×10^7 rows pgSphere’s index (3439 MB) exceeds **shared_buffers** while **skycell**’s (1071 MB) does not. 10^7 : the Gaia columns of Table 3; 5×10^7 : **bench/results-pg18/scale50/**. **Bold**: **skycell** faster, interval excluding 1.

radius	warm		cold	
	10^7	5×10^7	10^7	5×10^7
1''	0.70	0.78	0.79	0.82
10''	0.71	0.78	0.88	0.86
1'	0.74	0.86	0.96	0.98
6'	0.80	0.86	0.86	0.81
30'	0.75	0.75	0.69	0.53
1°	0.71	0.69	0.51	0.37
3°	0.67	0.73	0.51	0.42

Index size, build time, and what they cost at Gaia’s scale. Index size and build time are operational numbers worth carrying to the scale an archive runs at (Table 5). The cost per row is flat between 10^7 and 5×10^7 rows, so extrapolating to Gaia DR3’s 1.81×10^9 sources compares *deployments*: an archive wanting both cone searches and cross-matching carries pgSphere plus **q3c_join** (2.4 times faster than pgSphere at the latter, Sect. 6.5) today; **skycell** answers both from one, smaller index.

The memory column is the operational point: at DR3 scale a 64 GiB server holds **skycell**’s 38 GiB index with room to spare while the pgSphere-plus-Q3C pair needed for the same coverage wants 256 GiB. That factor-four gap is a procurement decision at these sizes, and it lands where our cold-cache measurements put **skycell** furthest ahead: warm **skycell** against cold pgSphere on the 5×10^7 -row corpus – the comparison a 64 GiB archive actually faces, since only the former’s index is resident there

Table 5: Index cost at 10 million rows and extrapolated to Gaia DR3 scale (1.81×10^9 sources, linear from the 5×10^7 measurement; per-row cost is flat to 0.5% between the two measured scales). Last column: smallest conventional server memory holding the index resident with room for the heap.

index	10^7 rows		at Gaia DR3 scale		
	MB	build (s)	GiB	build	mem GiB
Q3C	214	2.2	38	13 min	64
pgSphere	685	42.3	122	2.7 h	192
skycell	214	1.4	38	14 min	64
pgSphere + Q3C	—	—	160	2.9 h	256

– is one to two orders of magnitude apart at 30' and beyond.

This is an upper bound: the extrapolation is linear and validated only over a factor five in row count, covers the positional index alone (dwarfed in a real archive by the table and its other indexes), and a partitioned deployment needs only its active partitions resident. Since the two B-trees are the same size, the real saving is not needing the geometry index at all, which rests on **skycell** answering the region queries pgSphere is kept for (Sects. 6.4, 6.5).

The cost model: its parameter, its input, and what it is worth. Equation (2)’s one free parameter, c_{range} , need not be a parameter at all: it is a ratio of two costs PostgreSQL already models, one more index range (a B-tree descent) against one false-positive row (its index entry, heap tuple, exact predicate, and heap page amortised over the rows sharing it):

$$c_{\text{range}} = \frac{(\lceil \log_2 N \rceil + H + 1) 50 c_{\text{op}}}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + c_{\text{page}}/n_{\text{pp}}}, \quad (7)$$

with $n_{\text{pp}} = \text{reltuples}/\text{relpages}$. Every input is already at hand, so **skycell.range_cost** defaults to

evaluating it per relation (116 on the catalogues; 93 on a wider-row ObsCore relation). Placed in the same trials as the fixed constant it replaces, the two are indistinguishable, since the cost curve is flat (sweeping four decades moves the median time under 25% between 3 and 300): the derivation buys not speed but that the parameter stops being a number someone has to choose, and the order it picks is likewise close to optimal (about 3% against one order finer at 6') for the same reason.

Equation (7) is the rewrite's price, and knowingly high: it charges the descent's comparisons at 50 times what `btcostestimate` does and every page as a random read, which stands in for the per-range planning the rewrite incurs and Eq. (2) does not model. The custom scan plans no range, so it prices one as it pays for it: the descent as `btcostestimate` charges it, plus two pages – the leaf the descent lands on and the heap page where the scan stops running sequentially – each priced at $p = f 50 c_{\text{op}} + (1 - f) c_{\text{page}}$ by where it is expected to come from,

$$c_{\text{range}}^{\text{cs}} = \frac{\lceil \log_2 N \rceil c_{\text{op}} + 50(H + 1) c_{\text{op}} + 2p}{c_{\text{tuple}} + c_{\text{itup}} + k c_{\text{op}} + p/n_{\text{pp}}}, \quad (8)$$

with $f = \min(1, \text{shared_buffers}/\text{relpages})$ the fraction of the relation the buffer pool can hold: a page in the pool costs what touching an index page does, one outside it a random read. For the 10-million-row Gaia catalogue of the custom-scan runs (`random\_page\_cost` = 1.1, 120 rows per page) this is 34 rows with the relation in the pool and 87 with none of it there, against 110 from Eq. (7) for the same relation, so the custom scan cuts its coverings finer.

The model's input, $\hat{\rho}$ from Eq. (5), was called unbiased in an earlier version of this work – true of positions resampled from a density map with no structure below 0.11° , but not of real Gaia DR3 positions. Table 6 compares $\hat{\rho}$ with ρ counted from the same relation: within 5% at Baade's Window and a factor 1.3 at the Galactic pole (where the sample holds a few dozen rows), *low by a factor 10–100* in a globular cluster core, *high by a factor 3.7* at the Galactic centre, as the histogram resolves a cluster while a query sits inside it and averages it away once the query is larger (the failure Sect. 2.2 predicts, now measured on real positions). The estimate also moves between ANALYZE runs, which sample the relation afresh: at 47 Tuc the median ranges over 0.01–0.11 across ten. The consequence is smaller than the error: since $s^* \propto \sqrt{\rho}$, an order of magnitude in ρ is a factor three in cell size, and the cost curve is flat across much of that – the covering chosen in a cluster core, now through the custom scan, is within 1.05–1.53 of the best fixed order, against 0.98–1.31 where the estimate is accurate (an earlier run found 0.96–1.55, and 1.76 in a sample that put 47 Tuc a hundredfold low), and against 243–830 for a fixed

Table 6: The density estimate against the truth, real Gaia DR3 positions: $\hat{\rho}$ is what the cost model reads, ρ counted from the same relation, both in rows sr^{-1} , over four decades of surface density, radii 0.01° – 1° . $\hat{\rho}/\rho$: median over radii, then median over ten ANALYZE samples; *samples*: spread of that median across them; *range*: over all radii and samples. `bench/29_estimator.sql`, PostgreSQL 18.6, `bench/results-pg18/estimator/`.

field	$\hat{\rho}/\rho$	samples	range
ω Cen	0.08	0.06–0.09	0.05–0.54
47 Tuc	0.02	0.01–0.11	0.00–0.94
LMC	0.80	0.46–0.92	0.22–1.26
Baade's Window	0.96	0.92–0.98	0.45–1.19
Galactic pole	1.27	1.18–1.49	0.48–1.69
Galactic centre	3.71	3.41–3.80	1.95–5.20

order chosen badly. The error costs something in cluster cores at intermediate radii; it does not cost a wrong covering. The estimator should be judged by the decision it produces, not the number it reports.

The comparison with pgSphere confounds three things, separated on the real Gaia corpus: *heap clustering* is a large absolute effect but almost no differential one (ratio 0.64 clustered against 0.60 unclustered at 1°); *adaptive resolution* is worth a great deal (against every fixed order 4–13 the penalty for choosing badly is 243–830 $\times$, while the cost model lands within 1.02–1.58 of the best without knowing the answer); and *the index representation* is isolated by comparing against Q3C instead, both B-trees over an `int8` key differing only in covering strategy (Sect. 6.4, with the caveat of Sect. 6.4).

The service-level A/B, and where the crossover is. A benchmark of index operations is not a benchmark of an archive. We ran the same build inside `egernia` (Delli Veneri & Salgado 2026), a TAP 1.1 server, against its published 500 096-row ObsCore corpus: one database holding both indexes, two otherwise identical service processes differing only in how they translate CONTAINS, interleaved over six query classes at 1, 8 and 32 concurrent clients. Both targets pass the conformance and cross-server agreement gates on all eleven portable classes, identical rows through both, and the performance result is a tie (throughput ratios 0.96–1.06) although at the database level `skycell`, through the range rewrite of the first host, is 1.2–1.45 times *slower* per query: a cone over half a million rows costs a fraction of a millisecond either way against roughly 10 ms of CPU in the service's own Python, so the database cannot move throughput. That explanation predicts a crossover at larger relations, checked by building the same ObsCore-shaped relation (observations in groups of 128 sharing a field centre, clustered by collection and field) at three sizes and asking each the same 30 queries (Table 7). Field and

query centres are a stored set drawn once from the DR3 density map (`bench/data/`), and the relation at a given size is the same on every run. Earlier revisions of this table took the field centres from the first rows of whichever catalogue corpus was loaded; the resampled-Gaia corpus is generated in HEALPix order, so those rows were one patch of sky and every observation was packed into it (a 2° cone returned 1.2×10^5 rows at 10^7 , against 8×10^3 now), and the result moved with the corpus – the irreproducibility those revisions reported, not a property of either index. Like the synthetic columns of Table 3, it was measured with `skycell`’s custom scan (Sect. 6.4), and the range rewrite alongside, on PostgreSQL 18.6 in a 4-core, 15 GB cloud container, not the host of Table 2. The predicted crossover is there. The small cone (Q05) is 1.14 times slower than pgSphere at 5×10^5 rows and at parity by 10^7 ; the cone with catalogue cuts (Q07) is at parity up to 2×10^6 and 20% faster at 10^7 ; the degree-scale cone (Q06) is never slower and 25% faster at 10^7 , where pgSphere reads 939 buffer pages to `skycell`’s 591; the empty cone (Q12), a few hundredths of a millisecond either way, is faster at every size. Run with the range rewrite instead (in the same trials, `bench/results-pg18/crossover-stored/`), the same queries reach nearly the same ratio on Q06 at 10^7 (0.78), but the small cone stays 2.1–3.5 times slower and Q07 does not cross: for degree-scale cones the margin is the index, for the small ones it is the custom scan, which at 10^7 plans Q05 in 0.024 ms against the rewrite’s 0.086 and executes it in 0.091 ms against 0.168: the rewrite’s plan is a `BitmapOr` with an index descent and a bitmap per range, where the custom scan walks all its ranges into one bitmap. The three translations return identical rows for every query, size and run.

When the index no longer fits in memory. Tested directly in a 3 GiB container (768 MB shared buffers) holding a 22 GB, 20.5-million-row ObsCore-shaped relation (GiST index $1.51 \times$ the buffer pool, `skycell` $0.57 \times$): `skycell` answers 4.2–4.8 times faster over 3500 uniform 2° cones, serving 53% of its index accesses from the pool against pgSphere’s 9%. The 22 GB heap takes most of the pool regardless, so neither index is fully resident – this measures a smaller index winning a contested pool – and interleaving both translations in one pass instead of separate phases (each then leaving the heap warm for the other) puts the gap at $1.6 \times$ rather than 4.2 .

6.5 Crossmatches

Convex polygons of 0.05° – 2° take a median 0.281 ms on the Gaia corpus against pgSphere’s 0.767 ms and Q3C’s 1.864 ms, similarly on the designed corpus: a factor 2.7–4.4 either way, with pgSphere’s mean and

Table 7: Crossover on an ObsCore-shaped relation, randomized paired trials (30 stored centres, the same at every size; each method warmed on its own query first): total query time of `skycell`’s custom scan as the ratio to pgSphere (pgS.), pooled over two complete runs of 3 repetitions that differ only in trial order; median buffer pages at 10^7 rows. Q05: 0.15° cone (median 126 rows at 10^7); Q06: 2° (8350); Q07: 0.5° with calibration/time cuts (502); Q12: an empty $0.7''$ cone. Between the two runs the custom-scan ratio differs by at most 0.19 (Q07, 5×10^5), and by 0.02 or less at 10^7 except Q12 (0.12). PostgreSQL 18.6, 4-core cloud container (text). `bench/10.crossover.sql`, `bench/results-pg18/crossover-stored/`.

rows (10^6)	skycell / pgS. buffers, 10^7				
	0.5	2	10	pgS.	skycell
Q05	1.14	1.07	0.97	32	32
Q06	0.92	0.93	0.75	939	591
Q07	0.94	0.99	0.80	113	89
Q12	0.71	0.68	0.47	7	3

95th percentile far above its median where a polygon’s bounding volume is a poor fit.

Cross-matching a user’s uploaded target list is the operation the Gaia archive lists among its server-side capabilities (Salgado et al. 2017) and Q3C’s own speciality (`q3c_join`), so it deserves the cone searches’ protocol, with each method warmed on its own query immediately before timing to avoid a spurious I/O-ordering effect at these sizes (14 GB against 2 GB of `shared_buffers`). In `skycell` it is the same join written either way, `JOIN cat c ON skycell_cone(c.cell, c.ra, c.dec, p.ra, p.dec, r)`, answered by the range rewrite’s fixed slots or by the custom scan’s covering per probe (Sect. 2.3). `q3c_join` is the fastest established method, 2.4–2.6 times faster than pgSphere. At 10^7 rows (Table 8), the custom scan makes the join 25–29% faster than `q3c_join` and 3.2–3.7 times faster than pgSphere, returning the same rows as `q3c_join` in every trial; run through the rewrite in the same trials it is again not distinguishable from `q3c_join` (0.99–1.17). Written as `skycell_join`, the spelling a `q3c_join` user would write, over an expression index on `skycell_ang2cell(ra, dec)`, it is the same node at the same speed (0.96–0.97 of the `skycell_cone` form, not distinguishable), 27–33% faster than `q3c_join`. The difference is per-probe work, not the sky scanned: the rewrite’s plan is fixed at four range slots per probe, a `BitmapOr` whose every arm descends the index whether the probe’s covering filled it or not (12.6 buffer pages per probe in one block at $1''$), where the custom scan walks the probe’s own covering, 1.2 ranges on average, in one node (4.3 pages). The covering itself is no tighter – the sky it scans, as a multiple of the sky asked for, still peaks

Table 8: Cross-matching 200 000 probes against the 10^7 -row resampled-Gaia catalogue on the second host (Table 2): paired ratio over 16 trials (8 blocks of 25 000 probes, twice), 95% bootstrap interval. *Cone*: the join written with `skycell_cone` on the stored cell column; *join*: written as `skycell_join`, Q3C’s spelling, on an expression index; *custom*: answered by the custom scan; *slots*: through the range rewrite. **Bold**: interval excludes 1. `bench/11_xmatch.ab.sql`, `bench/results-pg18/xmatch/`.

	1''	10''
against q3c_join		
cone, custom	0.71 [0.64, 0.79]	0.75 [0.69, 0.86]
join, custom	0.67 [0.60, 0.77]	0.73 [0.63, 0.84]
cone, slots	0.99 [0.85, 1.15]	1.17 [0.94, 1.40]
against pgSphere		
cone, custom	0.27 [0.23, 0.30]	0.31 [0.24, 0.35]
join, custom	0.26 [0.23, 0.30]	0.31 [0.26, 0.33]

Table 9: Cross-matching at the radii the operation is used at, on **real Gaia DR3 positions** (10^7 , second host): geometric mean of the per-cell ratio of medians over three upload sizes (10^3 – 10^5) and two target distributions, 3 repetitions, method order random per cell. *Cone*, *join*: as in Table 8, through the custom scan. Below 1: `skycell` faster; **bold**: faster in every cell. `bench/18_xmatch_sweep.sql`, `bench/results-pg18/xmreal/`.

	vs q3c_join		vs pgSphere
radius	cone	join	join
0.5''	0.81	0.77	0.30
1''	0.77	0.76	0.30
1.5''	0.77	0.78	0.34
5''	0.84	0.83	0.35
all	0.80	0.79	0.32

around 105 near 0.25–1'', since the cost model won’t cut finer where extra ranges cost more than the area saved, and falls to 1.07 above 30°.

On real Gaia DR3 positions, at the radii cross-matching actually uses – optical 1–1.5'', radio up to 5'' – with three upload sizes and clustered or uniform targets (Table 9), the result holds: the custom scan’s join is 0.79 of `q3c_join` and 0.32 of `pgSphere` over all 24 cells, and faster than both in every one. Through the rewrite’s fixed slots the same join is 1.19 of `q3c_join`, the per-probe cost the custom scan removes. On the operation Q3C is built for, then, `skycell` is 20–33% faster than `q3c_join` on every corpus measured and 2.9–3.8 times faster than `pgSphere`, from the same index that answers the region queries of Sect. 6.4. Stored footprints follow the same pattern: the MOC-in-a-B-tree arrangement answers 200 474 point-in-region pairs in 0.76s against `pgSphere`’s 0.97s, with a larger, slower-to-build index since a region becomes several rows rather than one.

Table 10: Region-opclass wall-clock, real 50 000-row `fpr` catalogue, 500 probes/strategy: ratio of the `float4` box (`skyregion_box4_gist_ops`, now default) to `pgSphere`’s native GiST. Below 1: `skycell` faster.

strategy	box4 / pgSphere
<code>&&</code> (overlap)	0.12–0.17
<code>@></code> (region,point)	0.75–0.93
<code>@></code> (region,region)	0.55–0.70
<code><@</code> (region,region)	0.26–0.27

Estimates, plans and joins. A spatial predicate is rarely alone: three shapes (a cone with a catalogue cut, joined to an observation table, and a three-way join) through each method show `skycell`’s estimate advantage carrying over (median error 1.15–1.88 against 1.63–2.33 for Q3C and 2.50–3.00 for `pgSphere`), but the plans are the same – all three chose the same strategy in 134 of 135 queries. Better estimates are demonstrated, better plans are not: an accurate estimate is insurance against a query we did not write, and we have not shown that insurance paying off on one we did.

6.6 Regions

On isolated single-scale bands (0.05° – 1°), the `float4` box key (`skyregion_box4_gist_ops`, Sect. 3.1) beats `pgSphere`’s native GiST by 10–25% fewer buffers and 10–20% less time; on a mixed circle/polygon corpus the margin widens (`pgSphere` pays for two index descents per probe there, one per region kind), and on the real 50 000-row `fpr` catalogue, across all four region strategies (Table 10), the `float4` box wins every one – including point containment, the one strategy its own `float8` predecessor still loses to `pgSphere` on.

For points, the same shrink buys nothing: buffers are statistically indistinguishable from the unshrunk cap key at every radius from 1'' to 3°, and `pgSphere`’s native point GiST still wins by a stable 1.4–2.7× from 1'' through 30', converging within noise by 1° – unchanged by the shrink, since the structural reason region leaves could shrink safely does not hold for points (Sect. 3.1).

7 Conclusions

`skycell` keeps Q3C’s compact integer key and B-tree while answering the region queries `pgSphere` was built for, and treats a covering’s depth as a cost question – local source density against the price of an index range – rather than a constant. Measured paired against both, it beats each on its own ground. Against `pgSphere`, at the cone search, its custom scan is faster at every radius from 1'' to 3° warm (17–33% on the synthetic corpora, 4–31% on real Gaia DR3 positions) and faster or level cold (up to 2×), and it answers convex polygons 2.7–4.4× faster

(Sect. 6.4). Against Q3C, at the cross-match Q3C is built for, it is 20–33% faster than `q3c-join`, on re-sampled and real positions alike, and 2.9–3.8× faster than `pgSphere` (Sect. 6.5). It does so with an index a third the size of `pgSphere`’s (38 against 160 GiB at Gaia DR3 scale for the `pgSphere-plus-Q3C` pair covering the same ground), built thirty times faster, with row estimates good to a factor 1.2 rather than 2.5–3.0, from a cost model whose one parameter is derived from the relation’s own statistics rather than configured. Scale does not erode the advantage: at 50 million rows the custom scan stays 14–31% faster warm at every radius, and its cold lead for regions of 30’ and above widens, to a factor 2.7 at 1° (Sect. 6.4). For stored regions, a `GiST opclass` family with a `float4` box key beats every alternative tried, `pgSphere`’s native `GiST` included, on every corpus measured (Sect. 6.6).

The interface matters as much as the speed. `skycell` speaks ADQL’s own vocabulary with an operator form the planner answers from an index; its stored regions read and write as IVOA STC-S text with no translation step; and because `skyregion` is tagged by kind rather than split into a family of types, one column holds circles and polygons together, which `pgSphere` cannot. Neither established extension closes these gaps alone. One compact index serving points, regions and cross-matches through the language archives already use makes `skycell` worth considering as the default positional index of a PostgreSQL archive, and in particular as a TAP service’s geometry layer (Sect. 6.4).

The limits are specific. Every custom-scan number comes from one type of 4-vCPU cloud host (Table 2), where a cold read costs about 0.1 ms, a cache beneath the virtual machine rather than a disk, so the cold margins are likely conservative. On real positions the cold small cones are level rather than won (Sect. 6.4). On a small relation the small cone still favours `pgSphere` (1.14 at 5×10^5 rows, Table 7), and a TAP server dominated by its own Python leaves the choice immaterial. Better row estimates did not, on the join shapes we wrote, produce better plans.

What remains expensive is planning: a covering costs 0.015–0.12 ms even cached, against 0.006–0.008 ms for a `GiST` predicate needing no such step. Of three attempts to cut it, one paid off modestly: filling the per-order chord bounds lazily changes no covering and cuts planning by about 1% (95% bootstrap CI excluding 1). An SP-`GiST` class over the cell hierarchy shrinks the duplicate-position tolerance by an order of magnitude, so it stays exploratory. Letting the covering search try finer orders than Eq. (3)’s closed form (`skycell.probe_orders`, off by default) is worth 30–48% in *execution* time at several radii but reverses in *total* time against centres that do not repeat, since the extra classifications are paid at plan time on every query – a caution about measuring a cost-based index’s own overhead, not

only what it saves. The density synopsis is the other limit: an equi-depth histogram cannot see a cluster smaller than one of its buckets, and raising the statistics target does not help; the area guard bounds the damage, and a multi-order count map (`typanalyze`) is implemented as an option awaiting a measurement that shows it helps.

None of these limits touches the central result. For the queries archives serve most – cones, regions and cross-matches on catalogues large enough for the scan to dominate – a cost-based `HEALPix B-tree` is faster than the two extensions that carry that traffic today, smaller, and closer to the language the queries arrive in.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence. Both corpora are generated from seeds by the scripts recorded there, reproducible without transferring data; `bench/run.sh` and scripts 07–10 reproduce every table, and the TAP A/B of Sect. 6.4 is recorded there too.

Acknowledgements. This work has made use of data from the European Space Agency (ESA) mission *Gaia* (<https://www.cosmos.esa.int/gaia>), processed by the *Gaia* Data Processing and Analysis Consortium (DPAC, <https://www.cosmos.esa.int/web/gaia/dpac/consortium>), funded by national institutions participating in the *Gaia* Multilateral Agreement. Only per-cell source counts were used, to build the density field of Sect. 6.1.

The implementation, benchmark harness and analysis were developed with the assistance of Anthropic’s Claude Code. Because that bears on how the results should be read: the geometric predicates and covering are checked by the brute-force suites of Sect. 4 against independent computations over millions of adversarially placed points rather than expected outputs; every indexed answer in the regression suites must equal a sequential scan with the exact predicate; and the error of Sect. 2.2 was found by those tests rather than by inspection, the argument for relying on them. The benchmark harness was itself audited by reproducing its headline comparison under a different protocol (Sect. 6.4), which is how the difference between the two came to light.

References

- Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, *Astronomical Data Analysis Software and Systems XIII*, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225

- Delli Veneri, M., & Salgado, J. 2026, egeria: an IVOA TAP 1.1 server for the SKA Observatory, Zenodo, doi:10.5281/zenodo.22845690
- Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, *Astron. Comput.*, 7–8, 27
- Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, IVOA Recommendation: Table Access Protocol Version 1.1
- ESA 1997, The Hipparcos and Tycho Catalogues, ESA SP-1200
- Fernique, P., Boch, T., Donaldson, T., et al. 2014, IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0
- Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets, Microsoft Research Technical Report MSR-TR-2006-52
- Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124
- Ortiz, I., Lusted, J., Dowler, P., et al. 2008, IVOA Recommendation: Astronomical Data Query Language Version 2.0
- Rots, A. H. 2007, IVOA Recommendation: Space-Time Coordinate Metadata for the Virtual Observatory Version 1.33
- Salgado, J., González-Núñez, J., Gutiérrez-Sánchez, R., et al. 2017, *Astron. Comput.*, 21, 22
- Salgado, J., et al. 2020, in ASP Conf. Ser. 522, *Astronomical Data Analysis Software and Systems XXVII*, ed. P. Ballester, J. Ibsen, M. Solar, & K. Shortridge (San Francisco: ASP), 307